

ZOH O ANALYTICS

SQL Query Reference

Complete guide to writing effective SQL in Zoho Analytics —

Functions · Commands · Limitations · Workarounds · Best Practices

■ What's Covered	■ Key Focus	■■ You Will Learn
SELECT · WHERE · JOIN GROUP BY · HAVING Subqueries · CASE WHEN All built-in functions	Zoho-specific syntax Zoho vs standard SQL Zoho formula columns Data blending SQL	Write optimised queries Work around Zoho limits Debug common errors Build powerful reports

■ TABLE OF CONTENTS

Section 1	Zoho Analytics SQL — Overview & Architecture	3
Section 2	Core SQL Commands in Zoho	5
Section 3	String Functions — Complete Reference	9
Section 4	Numeric & Math Functions	13
Section 5	Date & Time Functions	16
Section 6	Aggregate & GROUP BY Functions	20
Section 7	Conditional Logic — CASE WHEN & IF	23
Section 8	Joins & Data Blending in Zoho	26
Section 9	Subqueries & Nested Queries	29
Section 10	Zoho Limitations & How to Overcome Them	32
Section 11	Query Optimisation & Best Practices	38
Section 12	Quick Reference Cheat Sheet	41

SECTION 1

ZOHIO ANALYTICS SQL

Overview & Architecture

What is Zoho Analytics SQL?

Zoho Analytics uses a subset of ANSI SQL called **ZOHIO SQL** — it looks and feels like standard SQL but has important differences. It is used in three places: **Query Tables** (custom SQL queries that act like virtual tables), **Formula Columns** (per-row computed columns), and **Aggregate Formulas** (summary metrics for reports).

Context	Where You Write SQL	Supports	Does NOT Support
Query Table	Setup > Create Query Table	SELECT, JOIN, WHERE, GROUP BY, subqueries	INSERT, UPDATE, DELETE, DDL
Formula Column	Right-click column > Add Formula Column	Row-level expressions, IF, CASE, string/date/math functions	Aggregate functions (SUM, COUNT etc.)
Aggregate Formula	Report > Add Formula	SUM, AVG, COUNT, custom metrics	Subqueries
Data Blending	Lookups / table joins	LEFT/INNER join between tables	FULL OUTER, CROSS JOIN

Key Differences from Standard SQL

Feature	Standard SQL	Zoho SQL	Impact
String quotes	Single ' or double "	Double " for aliases only; values use '	Low
Table names	Plain name	Exact name with spaces → wrap in quotes	Medium
Window functions	ROW_NUMBER, RANK, LAG etc.	NOT supported natively	HIGH — use workarounds
CTEs	WITH clause	NOT supported	HIGH — use subqueries
LIMIT / TOP	LIMIT n or TOP n	LIMIT n (MySQL style)	Low
NULL handling	IS NULL / COALESCE	IS NULL / IFNULL()	Low
Regex	REGEXP / SIMILAR TO	NOT supported	Medium
Stored procedures	Supported in RDBMS	NOT supported	High
INSERT/UPDATE	Supported	NOT supported in Query Tables	High
PIVOT	Varies by DB	NOT supported — use CASE WHEN	Medium

■ Zoho Analytics Query Tables use MySQL-compatible SQL syntax underneath. When in doubt, think MySQL 5.7 — most MySQL functions work in Zoho.

SECTION 2

CORE SQL COMMANDS

SELECT, WHERE, GROUP BY, ORDER BY & More

The Full SELECT Anatomy

Complete SELECT statement structure in Zoho SQL

```
SELECT
    column1,
    column2,
    expression AS alias_name,
    CASE WHEN condition THEN 'A' ELSE 'B' END AS category
FROM "Table Name"           -- quotes needed if name has spaces
JOIN "Another Table"
    ON "Table Name".key_col = "Another Table".key_col
WHERE condition1
    AND condition2
    OR  condition3
GROUP BY column1, column2
HAVING aggregate_condition
ORDER BY column1 ASC, column2 DESC
LIMIT 100;
```

SELECT — Selecting Columns

SELECT examples

```
-- All columns
SELECT * FROM "Sales Data";

-- Specific columns
SELECT "Customer Name", "Revenue", "Region" FROM "Sales Data";

-- Computed column
SELECT "Revenue" - "Cost" AS Profit FROM "Sales Data";

-- Removing duplicates
SELECT DISTINCT "Region" FROM "Sales Data";

-- Alias a table for shorter references
SELECT s."Customer Name", s."Revenue"
```

```
FROM "Sales Data" s;
```

WHERE — Filtering Rows

WHERE clause examples

```
-- Simple comparison
SELECT * FROM "Sales Data" WHERE "Revenue" > 10000;

-- Multiple conditions
SELECT * FROM "Sales Data"
WHERE "Region" = 'North'
    AND "Revenue" > 5000
    AND "Status" != 'Cancelled';

-- IN list
SELECT * FROM "Sales Data"
WHERE "Region" IN ('North', 'South', 'East');

-- NOT IN
SELECT * FROM "Sales Data"
WHERE "Status" NOT IN ('Cancelled', 'Returned');

-- BETWEEN (inclusive)
SELECT * FROM "Sales Data"
WHERE "Revenue" BETWEEN 5000 AND 20000;

-- LIKE pattern matching
SELECT * FROM "Sales Data"
WHERE "Customer Name" LIKE 'A%';      -- starts with A
-- WHERE "Email" LIKE '%@gmail.com';  -- ends with @gmail.com
-- WHERE "Code" LIKE 'A_C';           -- A, any one char, C

-- NULL checks
SELECT * FROM "Sales Data" WHERE "Discount" IS NULL;
SELECT * FROM "Sales Data" WHERE "Discount" IS NOT NULL;
```

GROUP BY & HAVING

GROUP BY with HAVING

```
-- Total revenue per region
SELECT "Region", SUM("Revenue") AS Total_Revenue
FROM "Sales Data"
GROUP BY "Region";

-- HAVING filters on aggregated values (not WHERE)
```

```
SELECT "Region", SUM("Revenue") AS Total_Revenue
FROM "Sales Data"
GROUP BY "Region"
HAVING SUM("Revenue") > 50000
ORDER BY Total_Revenue DESC;

-- Multi-level grouping
SELECT "Region", "Product Category",
       COUNT(*) AS Deals,
       AVG("Revenue") AS Avg_Revenue
FROM "Sales Data"
GROUP BY "Region", "Product Category"
ORDER BY "Region", Avg_Revenue DESC;
```

■ ■ In Zoho SQL, every column in SELECT that is NOT inside an aggregate function MUST appear in GROUP BY. Forgetting this causes 'Column not in GROUP BY' errors.

ORDER BY & LIMIT

Sorting and limiting results

```
-- Sort ascending (default)
SELECT * FROM "Sales Data" ORDER BY "Revenue";

-- Sort descending
SELECT * FROM "Sales Data" ORDER BY "Revenue" DESC;

-- Sort by multiple columns
SELECT * FROM "Sales Data"
ORDER BY "Region" ASC, "Revenue" DESC;

-- Top 10 rows
SELECT * FROM "Sales Data"
ORDER BY "Revenue" DESC
LIMIT 10;

-- Pagination (skip 20, get next 10)
SELECT * FROM "Sales Data"
ORDER BY "Revenue" DESC
LIMIT 10 OFFSET 20;
```

SECTION 3

STRING FUNCTIONS

Complete Reference with Examples

All Zoho String Functions

Function	Syntax	Description	Example
CONCAT	CONCAT(s1, s2, ...)	Join strings together	CONCAT('Hello', ' ', 'World')
LENGTH	LENGTH(str)	Number of characters	LENGTH('Zoho') → 4
UPPER	UPPER(str)	Convert to uppercase	UPPER('zoho') → 'ZOH0'
LOWER	LOWER(str)	Convert to lowercase	LOWER('ZOH0') → 'zoho'
TRIM	TRIM(str)	Remove leading & trailing spaces	TRIM(' hello ') → 'hello'
LTRIM	LTRIM(str)	Remove leading spaces only	LTRIM(' hi') → 'hi'
RTRIM	RTRIM(str)	Remove trailing spaces only	RTRIM('hi ') → 'hi'
SUBSTRING	SUBSTRING(str, pos, len)	Extract substring	SUBSTRING('Analytics',1,6) → 'Analyt'
LEFT	LEFT(str, n)	First n characters	LEFT('Analytics',4) → 'Anal'
RIGHT	RIGHT(str, n)	Last n characters	RIGHT('Analytics',4) → 'tics'
REPLACE	REPLACE(str, from, to)	Replace substring	REPLACE('ZohoZoho', 'Zoho', 'X') → 'XX'
INSTR	INSTR(str, substr)	Position of substring (1-based)	INSTR('Analytics', 'ly') → 5
LOCATE	LOCATE(substr, str)	Same as INSTR, arg order swapped	LOCATE('ly', 'Analytics') → 5
LPAD	LPAD(str, len, pad)	Left-pad to length	LPAD('5',4,'0') → '0005'
RPAD	RPAD(str, len, pad)	Right-pad to length	RPAD('Hi',5,'.') → 'Hi...'
REPEAT	REPEAT(str, n)	Repeat string n times	REPEAT('AB',3) → 'ABABAB'
REVERSE	REVERSE(str)	Reverse a string	REVERSE('Zoho') → 'ohoZ'
SPACE	SPACE(n)	n space characters	SPACE(3) → ' '
STRCMP	STRCMP(s1, s2)	Compare strings: -1, 0, or 1	STRCMP('a', 'b') → -1
CHAR	CHAR(n)	Character from ASCII code	CHAR(65) → 'A'
ASCII	ASCII(str)	ASCII value of first character	ASCII('A') → 65
CONCAT_WS	CONCAT_WS(sep, s1, s2, ...)	Concat with separator	CONCAT_WS('-', '2024', '01', '15')
FORMAT	FORMAT(num, dec)	Format number as string	FORMAT(1234567.8,2) → '1,234,567.80'
HEX	HEX(str or num)	Hexadecimal representation	HEX('A') → '41'

UNHEX	UNHEX(hex_str)	Convert hex back to string	UNHEX('41') → 'A'
-------	----------------	----------------------------	-------------------

String Function Recipes

Practical string query examples

```
-- Extract domain from email address

SELECT "Email",
       SUBSTRING("Email", INSTR("Email", '@') + 1) AS Domain
FROM "Contacts";

-- Build full name from first and last

SELECT CONCAT("First Name", ' ', "Last Name") AS Full_Name
FROM "Employees";

-- Standardise phone: keep only digits (workaround – nested REPLACE)

SELECT REPLACE(REPLACE(REPLACE(REPLACE("Phone", ' ', ''), '- ', ''), '( ', ''), ')', '' ) AS Clean_Phone
FROM "Contacts";

-- Pad invoice number to 8 digits

SELECT LPAD(CAST("Invoice ID" AS CHAR), 8, '0') AS Invoice_Code
FROM "Invoices";

-- Extract first word (first name) from full name

SELECT LEFT("Full Name", INSTR("Full Name", ' ') - 1) AS First_Word
FROM "Contacts";

-- Check if email contains 'gmail'

SELECT "Email",
       CASE WHEN INSTR(LOWER("Email"), 'gmail') > 0
            THEN 'Gmail User' ELSE 'Other' END AS Email_Provider
FROM "Contacts";
```

■ Case-Insensitive Searching

Zoho LIKE is case-insensitive by default. For explicit case-insensitive matching, use LOWER() or UPPER() on both sides: WHERE LOWER("Column") = LOWER('search_term')

SECTION 4

NUMERIC & MATH FUNCTIONS

Calculations & Number Formatting

All Zoho Numeric Functions

Function	Syntax	Description	Example
ROUND	ROUND(num, dec)	Round to decimal places	ROUND(3.14159, 2) → 3.14
FLOOR	FLOOR(num)	Round DOWN to integer	FLOOR(3.9) → 3
CEIL / CEILING	CEIL(num)	Round UP to integer	CEIL(3.1) → 4
ABS	ABS(num)	Absolute value	ABS(-42) → 42
MOD	MOD(num, div)	Remainder of division	MOD(17, 5) → 2
POWER / POW	POWER(base, exp)	Raise to a power	POWER(2, 10) → 1024
SQRT	SQRT(num)	Square root	SQRT(144) → 12
EXP	EXP(num)	e raised to the power	EXP(1) → 2.718...
LOG	LOG(num) / LOG(base, num)	Natural log or log base n	LOG(10, 100) → 2
LOG2	LOG2(num)	Log base 2	LOG2(8) → 3
LOG10	LOG10(num)	Log base 10	LOG10(1000) → 3
PI	PI()	Value of pi	PI() → 3.141592...
SIGN	SIGN(num)	Returns -1, 0, or 1	SIGN(-5) → -1
TRUNCATE	TRUNCATE(num, dec)	Truncate (no rounding)	TRUNCATE(3.99, 1) → 3.9
RAND	RAND()	Random float between 0 and 1	RAND() → 0.4823...
SIN/COS/TAN	SIN(num)	Trigonometric functions (radians)	SIN(PI()/2) → 1
DEGREES	DEGREES(rad)	Radians to degrees	DEGREES(PI()) → 180
RADIANS	RADIANS(deg)	Degrees to radians	RADIANS(180) → PI()
GREATEST	GREATEST(v1, v2, ...)	Maximum of a list of values	GREATEST(3, 7, 2) → 7
LEAST	LEAST(v1, v2, ...)	Minimum of a list of values	LEAST(3, 7, 2) → 2

Numeric Query Recipes

Numeric calculation examples

```
-- Profit margin percentage
SELECT "Product",
       ROUND(100.0 * ("Revenue" - "Cost") / "Revenue", 2) AS Margin_Pct
FROM "Sales Data";

-- Classify order size
```

```
SELECT "Order ID",
       ABS("Revenue" - "Target") AS Variance,
       SIGN("Revenue" - "Target") AS Direction -- 1=above, -1=below
FROM "Sales Data";

-- Discount bucket (round down to nearest 5)
SELECT FLOOR("Discount_Pct" / 5) * 5 AS Discount_Bucket
FROM "Orders";

-- Always avoid integer division – use 100.0 not 100
SELECT 100.0 * "Won Deals" / "Total Deals" AS Win_Rate -- correct
FROM "Pipeline";
```

SECTION 5

DATE & TIME FUNCTIONS

Working with Dates in Zoho Analytics

Date Functions Reference

Function	Syntax	Description	Example
NOW	<code>NOW()</code>	Current date and time	<code>NOW()</code> → '2024-03-15 14:30:00'
CURDATE	<code>CURDATE()</code>	Current date only	<code>CURDATE()</code> → '2024-03-15'
CURTIME	<code>CURTIME()</code>	Current time only	<code>CURTIME()</code> → '14:30:00'
DATE	<code>DATE(datetime)</code>	Extract date part	<code>DATE('2024-03-15 14:30')</code> → '2024-03-15'
TIME	<code>TIME(datetime)</code>	Extract time part	<code>TIME('2024-03-15 14:30')</code> → '14:30:00'
YEAR	<code>YEAR(date)</code>	Extract year	<code>YEAR('2024-03-15')</code> → 2024
MONTH	<code>MONTH(date)</code>	Extract month (1-12)	<code>MONTH('2024-03-15')</code> → 3
DAY / DAYOFMONTH	<code>DAY(date)</code>	Extract day of month	<code>DAY('2024-03-15')</code> → 15
DAYOFWEEK	<code>DAYOFWEEK(date)</code>	Day of week (1=Sun, 7=Sat)	<code>DAYOFWEEK('2024-03-15')</code> → 6 (Fri)
DAYOFYEAR	<code>DAYOFYEAR(date)</code>	Day number in year	<code>DAYOFYEAR('2024-03-15')</code> → 75
WEEKDAY	<code>WEEKDAY(date)</code>	0=Mon to 6=Sun	<code>WEEKDAY('2024-03-15')</code> → 4 (Fri)
WEEK	<code>WEEK(date)</code>	Week number (0-53)	<code>WEEK('2024-03-15')</code> → 11
QUARTER	<code>QUARTER(date)</code>	Quarter (1-4)	<code>QUARTER('2024-03-15')</code> → 1
HOURL	<code>HOUR(time)</code>	Extract hour (0-23)	<code>HOUR('14:30:00')</code> → 14
MINUTE	<code>MINUTE(time)</code>	Extract minutes	<code>MINUTE('14:30:45')</code> → 30
SECOND	<code>SECOND(time)</code>	Extract seconds	<code>SECOND('14:30:45')</code> → 45
DATE_FORMAT	<code>DATE_FORMAT(date, fmt)</code>	Format date as string	<code>DATE_FORMAT(date, '%b %Y')</code> → 'Mar 2024'
DATE_ADD	<code>DATE_ADD(date, INTERVAL n unit)</code>	Add to a date	<code>DATE_ADD(date, INTERVAL 30 DAY)</code>
DATE_SUB	<code>DATE_SUB(date, INTERVAL n unit)</code>	Subtract from a date	<code>DATE_SUB(date, INTERVAL 1 MONTH)</code>
DATEDIFF	<code>DATEDIFF(d1, d2)</code>	Days between two dates	<code>DATEDIFF('2024-03-15', '2024-01-01')</code> → 74
TIMEDIFF	<code>TIMEDIFF(t1, t2)</code>	Difference as time value	<code>TIMEDIFF('08:00', '06:30')</code> → '01:30:00'
STR_TO_DATE	<code>STR_TO_DATE(str, fmt)</code>	Parse string to date	<code>STR_TO_DATE('15-03-2024', '%d-%m-%Y')</code>

LAST_DAY	<code>LAST_DAY(date)</code>	Last day of that month	<code>LAST_DAY('2024-02-01') → '2024-02-29'</code>
DAYNAME	<code>DAYNAME(date)</code>	Name of day	<code>DAYNAME('2024-03-15') → 'Friday'</code>
MONTHNAME	<code>MONTHNAME(date)</code>	Name of month	<code>MONTHNAME('2024-03-15') → 'March'</code>
UNIX_TIMESTAMP	<code>UNIX_TIMESTAMP(date)</code>	Seconds since 1970-01-01	<code>UNIX_TIMESTAMP('2024-01-01')</code>
FROM_UNIXTIME	<code>FROM_UNIXTIME(ts)</code>	Unix timestamp to datetime	<code>FROM_UNIXTIME(1704067200)</code>

DATE_FORMAT Format Codes (Quick Reference)

Code	Meaning	Example Output
%Y	4-digit year	2024
%y	2-digit year	24
%m	Month 01-12	03
%c	Month 1-12	3
%M	Month name	March
%b	Abbreviated month	Mar
%d	Day 01-31	15
%e	Day 1-31	15
%D	Day with suffix	15th
%H	Hour 00-23	14
%h	Hour 01-12	02
%i	Minutes 00-59	30
%s	Seconds 00-59	00
%p	AM or PM	PM
%W	Weekday name	Friday
%a	Abbreviated weekday	Fri
%w	Day of week 0=Sun	5
%j	Day of year 001-366	075
%U	Week number 00-53	11
%Q	Quarter	Q1
%%	Literal % sign	%

Date Query Recipes

Practical date examples

```
-- Deals closed in current month

SELECT * FROM "Deals"

WHERE YEAR("Close Date") = YEAR(CURDATE())
```

```
AND MONTH("Close Date") = MONTH(CURDATE());

-- Revenue by quarter and year
SELECT YEAR("Close Date") AS Yr,
       QUARTER("Close Date") AS Qtr,
       SUM("Revenue") AS Revenue
FROM "Deals"
GROUP BY YEAR("Close Date"), QUARTER("Close Date")
ORDER BY Yr, Qtr;

-- Days since last activity
SELECT "Customer",
       DATEDIFF(CURDATE(), "Last Activity Date") AS Days_Since
FROM "Contacts"
ORDER BY Days_Since DESC;

-- Format date for display
SELECT DATE_FORMAT("Created Date", '%d %b %Y') AS Display_Date
FROM "Leads";

-- Start / end of current month
SELECT DATE_FORMAT(CURDATE(), '%Y-%m-01') AS Month_Start,
       LAST_DAY(CURDATE()) AS Month_End;

-- Age of a lead in days
SELECT "Lead Name",
       DATEDIFF(CURDATE(), "Created Date") AS Lead_Age_Days
FROM "Leads"
HAVING Lead_Age_Days > 30;
```

SECTION 6

AGGREGATE & GROUP BY FUNCTIONS

Summarising Data Effectively

Core Aggregate Functions

Function	Syntax	Description	Example
COUNT	<code>COUNT(*)</code> / <code>COUNT(col)</code>	Count rows or non-NULL values	<code>COUNT(*)</code> → total rows
COUNT DISTINCT	<code>COUNT(DISTINCT col)</code>	Count unique values	<code>COUNT(DISTINCT "Customer")</code>
SUM	<code>SUM(col)</code>	Total of numeric column	<code>SUM("Revenue")</code>
AVG	<code>AVG(col)</code>	Average of numeric column	<code>AVG("Deal Size")</code>
MIN	<code>MIN(col)</code>	Smallest value	<code>MIN("Close Date")</code>
MAX	<code>MAX(col)</code>	Largest value	<code>MAX("Revenue")</code>
GROUP_CONCAT	<code>GROUP_CONCAT(col SEPARATOR sep)</code>	Concatenate values in group	<code>GROUP_CONCAT("Tag" SEPARATOR ', ')</code>
VARIANCE	<code>VARIANCE(col)</code>	Statistical variance	<code>VARIANCE("Revenue")</code>
STD / STDDEV	<code>STD(col)</code>	Standard deviation	<code>STD("Revenue")</code>

Advanced GROUP BY patterns

```
-- Sales summary by region
SELECT
  "Region",
  COUNT(*) AS Total_Deals,
  COUNT(DISTINCT "Customer Name") AS Unique_Customers,
  SUM("Revenue") AS Total_Revenue,
  AVG("Revenue") AS Avg_Deal,
  MIN("Revenue") AS Smallest_Deal,
  MAX("Revenue") AS Largest_Deal
FROM "Sales Data"
GROUP BY "Region"
ORDER BY Total_Revenue DESC;

-- Conditional aggregation (pivot-style)
SELECT
  "Region",
  SUM(CASE WHEN "Status"='Won' THEN "Revenue" ELSE 0 END) AS Won_Revenue,
  SUM(CASE WHEN "Status"='Lost' THEN "Revenue" ELSE 0 END) AS Lost_Revenue,
```

```
        COUNT(CASE WHEN "Status"='Won' THEN 1 END) AS Won_Count
FROM "Deals"
GROUP BY "Region";

-- List all products per customer (GROUP_CONCAT)
SELECT "Customer Name",
       GROUP_CONCAT("Product" ORDER BY "Product" SEPARATOR ' | ') AS Products_Bought
FROM "Orders"
GROUP BY "Customer Name";

-- HAVING: only regions with 10+ deals
SELECT "Region", COUNT(*) AS Deals
FROM "Sales Data"
GROUP BY "Region"
HAVING COUNT(*) >= 10
ORDER BY Deals DESC;
```

■ Remember: WHERE filters rows BEFORE aggregation. HAVING filters AFTER aggregation. Use WHERE for raw column conditions, HAVING for conditions on SUM/COUNT/AVG etc.

SECTION 7

CONDITIONAL LOGIC

CASE WHEN, IF, IFNULL & COALESCE

CASE WHEN — The Swiss Army Knife

CASE WHEN patterns

```
-- Searched CASE (most flexible)

SELECT "Deal Name",
       "Revenue",
       CASE
         WHEN "Revenue" >= 100000 THEN 'Enterprise'
         WHEN "Revenue" >= 25000  THEN 'Mid-Market'
         WHEN "Revenue" >= 5000   THEN 'SMB'
         ELSE 'Micro'
       END AS Deal_Segment
FROM "Deals";

-- Simple CASE (equality checks)

SELECT "Region",
       CASE "Region"
         WHEN 'North' THEN 'Northern Zone'
         WHEN 'South' THEN 'Southern Zone'
         ELSE 'Other Zone'
       END AS Zone
FROM "Sales Data";

-- CASE in aggregate (conditional count)

SELECT COUNT(CASE WHEN "Status" = 'Won' THEN 1 END) AS Won,
       COUNT(CASE WHEN "Status" = 'Lost' THEN 1 END) AS Lost,
       COUNT(CASE WHEN "Status" = 'Open' THEN 1 END) AS Open
FROM "Deals";

-- Nested CASE

SELECT "Customer",
       CASE WHEN "Region" = 'North'
         THEN CASE WHEN "Revenue" > 50000 THEN 'North-High'
         ELSE 'North-Low' END
```

```
        ELSE 'Other'
    END AS Segment
FROM "Sales Data";
```

IF, IFNULL & NULLIF

IF and NULL-handling functions

```
-- IF(condition, true_val, false_val)
SELECT "Deal Name",
       IF("Revenue" > 10000, 'High Value', 'Standard') AS Priority
FROM "Deals";

-- Nested IF (use CASE WHEN for readability beyond 2 levels)
SELECT IF("Score" >= 80, 'A', IF("Score" >= 60, 'B', 'C')) AS Grade
FROM "Assessments";

-- IFNULL(expr, fallback) – replace NULL with a default
SELECT "Customer",
       IFNULL("Discount", 0) AS Discount_Applied
FROM "Orders";

-- NULLIF(a, b) – returns NULL if a = b (useful to avoid division by zero)
SELECT "Revenue" / NULLIF("Units", 0) AS Revenue_Per_Unit
FROM "Sales Data";

-- COALESCE – first non-NULL value from a list
SELECT COALESCE("Mobile", "Phone", "Alternate Phone", 'No Contact') AS Best_Phone
FROM "Contacts";
```

■ COALESCE is the most readable way to handle multiple fallback values. Prefer COALESCE over nested IFNULL for chains of 3 or more columns.

SECTION 8

JOINS & DATA BLENDING

Combining Tables in Zoho Analytics

JOIN Types Supported

JOIN Type	Returns	Zoho Support	Syntax
INNER JOIN	Only matching rows in BOTH tables	■ Yes	... JOIN ... ON ...
LEFT JOIN	ALL rows from left + matches from right	■ Yes	... LEFT JOIN ...
RIGHT JOIN	ALL rows from right + matches from left	■ Yes	... RIGHT JOIN ...
FULL OUTER JOIN	ALL rows from both tables	■ No	Workaround needed
CROSS JOIN	Every combination of rows	■■ Limited	Use carefully
SELF JOIN	Table joined to itself	■ Yes	Alias same table twice

JOIN examples

```
-- INNER JOIN: customers who have placed orders
SELECT c."Customer Name", o."Order Date", o."Amount"
FROM "Customers" c
JOIN "Orders" o ON c."Customer ID" = o."Customer ID";

-- LEFT JOIN: all customers, with order count (0 if none)
SELECT c."Customer Name",
       COUNT(o."Order ID") AS Order_Count,
       COALESCE(SUM(o."Amount"), 0) AS Total_Spent
FROM "Customers" c
LEFT JOIN "Orders" o ON c."Customer ID" = o."Customer ID"
GROUP BY c."Customer Name";

-- Multi-table join
SELECT c."Customer Name", p."Product Name", o."Amount", o."Order Date"
FROM "Orders" o
JOIN "Customers" c ON o."Customer ID" = c."Customer ID"
JOIN "Products" p ON o."Product ID" = p."Product ID"
ORDER BY o."Order Date" DESC;

-- SELF JOIN: manager-employee relationship
```

```
SELECT e."Name" AS Employee, m."Name" AS Manager
FROM "Employees" e
LEFT JOIN "Employees" m ON e."Manager ID" = m."Employee ID";

-- FULL OUTER JOIN workaround (LEFT + RIGHT UNION)

SELECT c."Customer ID", o."Order ID"
FROM "Customers" c LEFT JOIN "Orders" o ON c."Customer ID"=o."Customer ID"
UNION
SELECT c."Customer ID", o."Order ID"
FROM "Customers" c RIGHT JOIN "Orders" o ON c."Customer ID"=o."Customer ID";
```

■ Data Blending vs Query Table Joins

Zoho has two ways to combine tables: (1) Table Lookups — point-and-click joins for reports, no SQL needed; (2) Query Tables — full SQL JOIN syntax. For complex multi-table logic always use a Query Table. For simple reporting, Table Lookups are faster to set up.

SECTION 9

SUBQUERIES & NESTED QUERIES

Advanced Query Composition

Types of Subqueries in Zoho

Subquery in WHERE (scalar)

```
-- Customers who spent more than average
SELECT "Customer Name", SUM("Amount") AS Total
FROM "Orders"
GROUP BY "Customer Name"
HAVING SUM("Amount") > (
    SELECT AVG(customer_total)
    FROM (
        SELECT SUM("Amount") AS customer_total
        FROM "Orders"
        GROUP BY "Customer ID"
    ) AS sub
);
```

Subquery in FROM (derived table)

```
-- Top region by average deal size
SELECT r."Region", r.Avg_Deal
FROM (
    SELECT "Region", AVG("Revenue") AS Avg_Deal
    FROM "Deals"
    GROUP BY "Region"
) AS r
ORDER BY r.Avg_Deal DESC
LIMIT 1;
```

Subquery with IN / NOT IN

```
-- Customers who have NEVER placed an order
SELECT "Customer Name"
FROM "Customers"
WHERE "Customer ID" NOT IN (
    SELECT DISTINCT "Customer ID" FROM "Orders"
);
```

```
-- Products ordered in January
SELECT "Product Name"
FROM "Products"
WHERE "Product ID" IN (
    SELECT DISTINCT "Product ID"
    FROM "Orders"
    WHERE MONTH("Order Date") = 1
);
```

■ ■ Zoho does NOT support CTEs (WITH clause). Every subquery must be written inline as a derived table in FROM, or as a scalar subquery in WHERE/HAVING. For complex logic, break it into multiple Query Tables and join them.

Emulating a CTE using nested Query Tables

```
-- Step 1: Create Query Table 'Monthly_Revenue'
SELECT YEAR("Date") AS Yr, MONTH("Date") AS Mo, SUM("Amount") AS Revenue
FROM "Sales"
GROUP BY YEAR("Date"), MONTH("Date");

-- Step 2: Create a second Query Table that references the first
SELECT m1.Yr, m1.Mo, m1.Revenue,
       m1.Revenue - m2.Revenue AS MoM_Change
FROM "Monthly_Revenue" m1
LEFT JOIN "Monthly_Revenue" m2
    ON m1.Yr * 12 + m1.Mo = m2.Yr * 12 + m2.Mo + 1;
```

SECTION 10

LIMITATIONS & WORKAROUNDS

Overcoming Zoho SQL Restrictions

Zoho Analytics SQL is powerful but intentionally restricted compared to full RDBMS SQL. This section documents every significant limitation and the best workaround for each.

■ Known Limitations & Their Solutions

■ No Window Functions (ROW_NUMBER, RANK, LAG, LEAD)

Limitation: Zoho SQL does not support OVER() window functions — the most common gap developers hit.

■ **Workaround:** Use a self-join or a separate Query Table to emulate ranking.

```
-- Emulate ROW_NUMBER: rank deals by revenue per region
SELECT a."Deal ID", a."Region", a."Revenue",
       COUNT(b."Deal ID") + 1 AS Row_Num
FROM "Deals" a
LEFT JOIN "Deals" b
      ON a."Region" = b."Region"
      AND b."Revenue" > a."Revenue"
GROUP BY a."Deal ID", a."Region", a."Revenue"
ORDER BY a."Region", Row_Num;

-- Emulate LAG (previous row): use a self-join on row number
-- First build a numbered Query Table, then self-join on rn = rn-1
```

■ No CTE / WITH Clause

Limitation: You cannot write WITH monthly AS (...) SELECT ... in Zoho.

■ **Workaround:** Create intermediate results as separate named Query Tables, then join them.

```
-- Instead of a CTE, create Query Table 'Dept_Avg':
SELECT "Department", AVG("Salary") AS Avg_Salary
FROM "Employees"
GROUP BY "Department";

-- Then in another Query Table, reference 'Dept_Avg':
SELECT e."Name", e."Salary", d.Avg_Salary,
       e."Salary" - d.Avg_Salary AS Diff_From_Avg
FROM "Employees" e
JOIN "Dept_Avg" d ON e."Department" = d."Department";
```

■ No FULL OUTER JOIN

Limitation: Zoho only supports INNER, LEFT, and RIGHT joins. FULL OUTER JOIN throws an error.

■ **Workaround:** Use UNION of a LEFT JOIN and RIGHT JOIN to get all rows from both tables.

```
SELECT a.id, a.name, b.score
FROM "TableA" a LEFT JOIN "TableB" b ON a.id = b.id
UNION
SELECT a.id, a.name, b.score
FROM "TableA" a RIGHT JOIN "TableB" b ON a.id = b.id;
```

■ No PIVOT / CROSSTAB Syntax

Limitation: There is no PIVOT keyword in Zoho SQL.

■ **Workaround:** Use conditional aggregation with CASE WHEN to create pivot-style output.

```
SELECT "Region",
       SUM(CASE WHEN QUARTER("Date")=1 THEN "Revenue" ELSE 0 END) AS Q1,
       SUM(CASE WHEN QUARTER("Date")=2 THEN "Revenue" ELSE 0 END) AS Q2,
       SUM(CASE WHEN QUARTER("Date")=3 THEN "Revenue" ELSE 0 END) AS Q3,
       SUM(CASE WHEN QUARTER("Date")=4 THEN "Revenue" ELSE 0 END) AS Q4
FROM "Sales"
GROUP BY "Region";
```

■ No REGEX Support

Limitation: REGEXP and SIMILAR TO are not available. Pattern matching is limited to LIKE.

■ **Workaround:** Chain INSTR/SUBSTRING/REPLACE for complex pattern logic, or pre-clean data in Zoho Flow.

```
-- Check if a string starts with a digit (workaround)
SELECT * FROM "Data"
WHERE SUBSTRING("Code",1,1) IN ('0','1','2','3','4','5','6','7','8','9');

-- Extract digits only (nested REPLACE for each non-digit character)
-- For heavy text processing, use a formula column outside the query
```

■ No INSERT / UPDATE / DELETE in Query Tables

Limitation: Query Tables are read-only views. You cannot modify underlying data through SQL.

■ **Workaround:** Use Zoho Deluge scripts, Zoho Flow automations, or the API to write data. For transformations, create Formula Columns on the source table.

■ No Stored Procedures or Functions

Limitation: You cannot define custom SQL functions or stored procedures.

■ **Workaround:** Break complex logic into multiple Query Tables (like pipeline steps). Use Zoho Analytics Formula Columns for row-level logic.

■ No GROUP_CONCAT ORDER BY in Older Versions

Limitation: Some Zoho versions do not support ORDER BY inside GROUP_CONCAT.

■ **Workaround:** Sort the subquery/table before GROUP_CONCAT, or use a subquery to pre-sort.

```
-- Pre-sort with a subquery, then GROUP_CONCAT
SELECT "Customer ID",
       GROUP_CONCAT("Product" SEPARATOR ', ' ) AS Products
FROM (
  SELECT "Customer ID", "Product"
  FROM "Orders"
  ORDER BY "Customer ID", "Product"
) AS sorted
GROUP BY "Customer ID";
```

■ LIMIT 100,000 Row Cap on Query Tables

Limitation: Zoho limits Query Table result sets to 100,000 rows. Queries exceeding this are truncated.

■ **Workaround:** Always filter with WHERE, aggregate with GROUP BY, and use date range filters to keep result sets well under the limit. Design reports to summarise, not list raw rows.

■ No Dynamic SQL or Variables

Limitation: You cannot use SET @var or execute dynamic SQL strings.

■ **Workaround:** Use CASE WHEN with hard-coded thresholds, or pass filter values through Zoho Dashboard filter widgets which inject WHERE conditions at runtime.

■ Date Arithmetic Returns Days, Not Months/Years

Limitation: DATEDIFF always returns days. There is no native MONTHS_BETWEEN function.

■ **Workaround:** Divide by 30.44 for approximate months, or use YEAR/MONTH functions for exact periods.

```
-- Approximate months between two dates
SELECT ROUND(DATEDIFF("End Date", "Start Date") / 30.44, 1) AS Approx_Months
FROM "Projects";

-- Exact year+month difference
SELECT (YEAR("End Date") - YEAR("Start Date")) * 12
      + (MONTH("End Date") - MONTH("Start Date")) AS Exact_Months
FROM "Projects";
```

SECTION 11

QUERY OPTIMISATION

Best Practices for Fast, Reliable Queries

Performance Best Practices

■ Always Filter Early

Put WHERE clauses before GROUP BY to reduce the rows the engine processes. Filter on indexed columns (like Date, ID) first — Zoho handles these fastest.

■ Avoid SELECT *

Always list only the columns you need. SELECT * transfers all columns across the network and prevents Zoho from optimising the read. Name your columns explicitly.

■ Use LIMIT During Development

While writing or debugging a query, add LIMIT 100 to get fast feedback. Remove or increase the limit only when you are satisfied with the logic.

■ Aggregate Before Joining

If you need totals per customer, aggregate in a subquery first, then join. Never join raw tables first and aggregate later — it creates a much larger intermediate set.

■ Prefer JOIN over Subquery WHERE IN

WHERE col IN (SELECT ...) scans the subquery for every outer row. A JOIN is usually faster and more readable for the same logic.

■ Avoid Functions on Filtered Columns

WHERE YEAR("Date") = 2024 prevents index use. Prefer: WHERE "Date" BETWEEN '2024-01-01' AND '2024-12-31' — this is range-friendly.

■ Break Complex Queries into Query Tables

A query with 5+ levels of nesting is hard to maintain. Create named Query Tables for intermediate steps — they are reusable, testable, and much easier to debug.

■ Handle NULLs Explicitly

Always use IFNULL() or COALESCE() on columns that may be NULL before arithmetic. NULL + anything = NULL, and NULL in COUNT/SUM is silently excluded.

■ Use Aliases for Readability

Always alias aggregations: SUM(Revenue) AS Total_Revenue. Use short table aliases in JOINS: FROM "Customers" c. Avoid aliasing with reserved words like ORDER, GROUP, TABLE.

■ Integer Division Trap

$10 / 3 = 3$ (integer), not 3.33. Always cast to decimal: $10.0 / 3$ or `CAST(numerator AS DECIMAL) / denominator` for accurate percentage calculations.

Debugging Checklist

Error / Symptom	Likely Cause	Fix
'Column not in GROUP BY'	Selected column not in GROUP BY and not aggregated	Add column to GROUP BY or wrap in AGG function
Wrong row count	Cartesian product from missing JOIN condition	Check all ON conditions are correct
All NULLs in result	JOIN key mismatch (type or case)	Check both keys have same data type; use CAST if needed
Date filter not working	Date stored as text, not date type	Use STR_TO_DATE() to convert, or fix column type
Zero for percentage	Integer division	Use $100.0 * \text{numerator} / \text{denominator}$
Slow query	No filter, SELECT *, large table	Add WHERE, list columns, use LIMIT
Truncated results	Over 100K row limit	Add GROUP BY / filter to reduce rows
Syntax error near 'WITH'	CTE not supported	Replace WITH by using derived table or separate Query Table
Blank report	Query returns 0 rows	Test query alone, check WHERE conditions aren't too restrictive

SECTION 12

QUICK REFERENCE CHEAT SHEET

Everything on One Page

■ SELECT Statement Skeleton

```

SELECT col1, col2, AGG(col3) AS alias
FROM "Table" t1
JOIN "Table2" t2 ON t1.key = t2.key
WHERE condition
GROUP BY col1, col2
HAVING AGG(col3) > value
ORDER BY col1 DESC
LIMIT 100;

```

■ Most Used String Functions

Task	Function
Join strings	CONCAT(a, b, c) or CONCAT_WS(sep, a, b)
Length	LENGTH(str)
Upper/Lower	UPPER(str) / LOWER(str)
Extract part	SUBSTRING(str, start, len) LEFT(str,n) RIGHT(str,n)
Find position	INSTR(str, search)
Replace	REPLACE(str, from, to)
Trim spaces	TRIM(str) / LTRIM / RTRIM
Pad	LPAD(str, len, pad) / RPAD
Format number	FORMAT(num, decimals)

■ Most Used Date Functions

Task	Function
Current date/time	CURDATE() / NOW()
Extract parts	YEAR(d) / MONTH(d) / DAY(d) / QUARTER(d) / WEEK(d)
Days between	DATEDIFF(d1, d2)
Add/subtract	DATE_ADD(d, INTERVAL n DAY/MONTH/YEAR)
Format for display	DATE_FORMAT(d, '%d %b %Y')
Last day of month	LAST_DAY(date)

Day/month name	DAYNAME(d) / MONTHNAME(d)
Parse string to date	STR_TO_DATE(str, '%d-%m-%Y')

■ Conditional Functions

Function	Use For
CASE WHEN c THEN v ELSE d END	Multi-branch logic, segmentation, pivoting
IF(cond, true, false)	Simple two-branch logic
IFNULL(expr, default)	Replace NULL with a fallback value
NULLIF(a, b)	Return NULL when a = b (avoid division by zero)
COALESCE(a, b, c, ...)	First non-NULL value from a list

■ Zoho Limitations at a Glance

Feature	Status	Workaround
Window functions (RANK, LAG etc.)	■ Not supported	Self-join counting pattern
CTE / WITH clause	■ Not supported	Chain of Query Tables
FULL OUTER JOIN	■ Not supported	LEFT JOIN UNION RIGHT JOIN
PIVOT	■ Not supported	CASE WHEN conditional aggregation
REGEXP	■ Not supported	INSTR + SUBSTRING + REPLACE chains
INSERT/UPDATE/DELETE	■ Not supported	Zoho Deluge / Flow / API
Stored procedures	■ Not supported	Multiple Query Tables
Dynamic SQL / variables	■ Not supported	Dashboard filter widgets
Row limit	■■ 100K rows max	Filter + GROUP BY to reduce rows
CTEs	■ Not supported	Separate named Query Tables

You're Ready to Query Zoho Like a Pro! ■

Keep this guide open while you write queries. When you hit a limitation, check Section 10 for the workaround. When you need a function, check Sections 3–6.

— *Your Senior Developer*